Combating Missed Recalls in E-commerce Search: A CoT-Prompting Testing Approach

Shengnan Wu
School of Computer Science, Fudan University
Shanghai, China
snwu19@fudan.edu.cn

Yongxiang Hu
School of Computer Science, Fudan University
Shanghai, China
yongxianghu23@m.fudan.edu.cn

Yingchuan Wang
School of Computer Science, Fudan University
Shanghai, China
yingchuanwang23@m.fudan.edu.cn

Jiazhen Gu
School of Computer Science, Fudan University
Shanghai, China
jiazhengu@cuhk.edu.hk

Jin Meng
Meituan
Beijing, China
mengjin02@meituan.com

Liujie Fan
Meituan
Beijing, China
fanliujie@meituan.com

Zhongshi Luan
Meituan
Beijing, China
luanzhongshi@meituan.com

Xin Wang
School of Computer Science, Fudan University
Shanghai, China
xinw@fudan.edu.cn

Yangfan Zhou
School of Computer Science, Fudan University
Shanghai, China
zyf@fudan.edu.cn

ABSTRACT

Search components in e-commerce apps, often complex AI-based systems, are prone to bugs that can lead to missed recalls—situations where items that should be listed in search results aren't. This can frustrate shop owners and harm the app's profitability. However, testing for missed recalls is challenging due to difficulties in generating user-aligned test cases and the absence of oracles. In this paper, we introduce mrDetector, the first automatic testing approach specifically for missed recalls. To tackle the test case generation challenge, we use findings from how users construct queries during searching to create a CoT prompt to generate user-aligned queries by LLM. In addition, we learn from users who create multiple queries for one shop and compare search results, and provide a test oracle through a metamorphic relation. Extensive experiments using open access data demonstrate that mrDetector outperforms all baselines with the lowest false positive ratio. Experiments with real industrial data show that mrDetector discovers over one hundred missed recalls with only 17 false positives.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**; *Maintaining software*; • **Human-centered computing** → Usability testing.

KEYWORDS

Metamorphic Testing, Search Components, LLM

ACM Reference Format:
Shengnan Wu, Yongxiang Hu, Yingchuan Wang, Jiazhen Gu, Jin Meng, Liujie Fan, Zhongshi Luan, Xin Wang, and Yangfan Zhou. 2024. Combating Missed Recalls in E-commerce Search: A CoT-Prompting Testing Approach. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion '24), July 15–19, 2024, Porto de Galinhas, Brazil.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3663529.3663842

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FSE Companion '24, July 15–19, 2024, Porto de Galinhas, Brazil

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0658-5/24/07
https://doi.org/10.1145/3663529.3663842

1 INTRODUCTION

Search components play an important role in modern e-commerce platforms (*e.g.*, Amazon [39] and Yelp [46]). In such platforms, shop owners present products/services their shops provide to customers, who discover products/services by searching on the platforms. The search components of such platforms are usually complicated AI-based retrieval systems that take user-generated queries as input and search results, *i.e.* lists of entries (typically shops or products), as output. Apart from the user-generated queries, user preferences and searching contexts also affect the search results. Bridging users, shop owners and products for sale together, those search components cast a non-negligible impact on the revenue-making and success of e-commerce apps [14].

Complication is a notorious cause of bugs [40]. As complicated AI-based retrieval systems, search components in e-commerce apps are not bug-free. Existing work mainly focuses on false recalls, *i.e.*, the situations where an entry appears in the search result but should not be recalled according to algorithmic and business logic [20, 26, 30, 34, 45, 55, 57, 64, 67]. Those false recalls cause search results to be unstable, irrelevant, and inconsistent with user preferences. However, little research attention has been paid to situations where an entry should be recalled according to algorithmic and business

logic but does not appear in the search result (*i.e.*, the *missed* recalls). Missed recalls bring quite negative impacts. For example, customers may be dismayed to see their favorite diner neglected by the search component. Shop owners may be dissatisfied to see their shops not being presented to customers. As a result, they may decide to switch to competitor platforms, and eventually, the platform's profitability will suffer.

In real industrial settings, missed recalls are not rare. Take *M-App*, a prevalent e-commerce app developed by Meituan, one of the largest online shopping provides as an example. In the second quarter of 2023, over 20 percent of problematic search results are confirmed to be associated with missed recalls. Currently, missed recalls are typically found by feedback from shop owners . This post-mortem way of missed-recall-discovery has two major drawbacks. Firstly, it heavily relies on manual efforts and can only discover a limited number of missed recalls. Secondly, it is accompanied by complaints from shop owners who have found their shops can not be displayed to customers normally. Hence, an approach to detect missed recalls automatically and proactively is necessary.

Nevertheless, it is non-trivial to test a search component automatically and proactively targeting missed recalls. Two major challenges remain unsolved. Firstly, it is far from easy to generate **realistic** test cases automatically (challenge 1). In this scenario, test cases are user-generated natural language queries. How users express their shopping needs and the search context, which are subjective and ambiguous, should be involved in the generation process. Without realistic test cases, we will only end up with defects induced by edge cases that users seldom input into a search component in an e-commerce app. In the meantime, the **oracle** problem awaits (challenge 2). Unlike false recalls that stand out in the search results naturally, missed recalls are more salient and user-oriented. Specifically, it is the users, instead of the e-commerce app that hold the ultimate standards in judging missed recalls. Those user standards are subjective, perceptual and may vary among different users. Hence, given a specific case, it is accomplishable by human effort to judge whether there is a missed recall. However, it is challenging to formulate the way of human judgment into a set of automatic algorithms.

The user-oriented nature of missed recalls inspires us to analyze historical track of missed recalls, learn from the handling procedure and design our approach accordingly. We present missed recall Detector (mrDetector), which relies on a metamorphic relation to provide a test oracle and leverages the power of the large Language Model (LLM) to generate realistic test cases. The rationale behind this is that subjective and ambiguous domain knowledge about interaction and need expression are implicitly obtained by LLMs via a large training corpus. We also use the findings from analyzing historical missed recalls to guide prompt engineering and ensure context-compatible test cases. To our knowledge, mrDetector is the first testing approach targeting missed recalls in search components of e-commerce apps. It can also be generalized to other e-commerce apps besides *M-App*.

Extensive evaluation is conducted to compare the performance of mrDetector in different settings against multiple baselines. Experiments with open data show that mrDetector outperforms all baseline methods with the lowest false positive ratio. The ablation study revealed that an LLM with a larger parameter size, the customized chain of thought (CoT) prompt mimicking how general users construct queries and an LLM based validation step benefit the performance of mrDetector. We also deployed mrDetector in the field to detect real-world online missed recalls for *M-App*. In the real-world experience, we have detected over one hundred missed recalls with only 17 false positives.

The contributions of this paper are summarized as follows:

- To the best of our knowledge, we are the first to conduct a comprehensive study into missed recalls of search components in the e-commerce setting, which can shed light on future work of evaluating and optimizing complicated AI-based search components.
- We present mrDetector, a testing approach targeting missed recalls of search components in e-commerce apps. It leverages LLM to overcome the challenge of realistic test cases and a metamorphic relation to overcome the challenge of the oracle problem. It provides references to testing work involving the subjectivity of human and faced with the oracle problem.
- Experiments with open data illustrate that each major component benefits the performance of mrDetector, outperforming all baseline methods with the lowest false positive ratio. Experiments with industrial data show the handiness of mrDetector in the real industrial setting, with over 100 missed recalls found and over half of them estimated reproducible.

2 BACKGROUND AND MOTIVATION

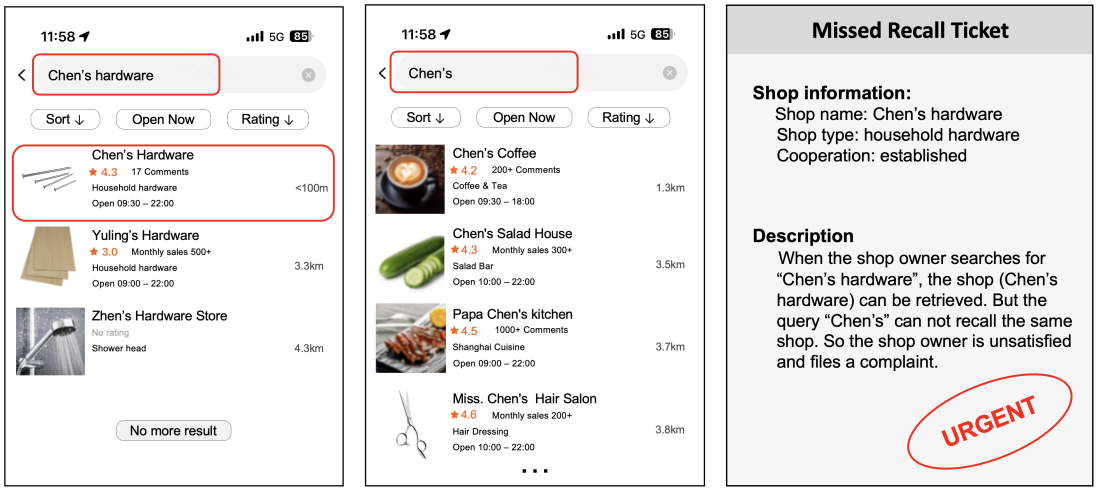
The AI-based search components play an important role. According to the annual report[1] of Meituan, the *M-App* has over 600 million active users and has established business cooperation with over 9 million shops. As users rely on the search component to discover what to buy [51] and shops rely on the search component to complete orders, bugs in search components not only degrade user experience, but also hinder the profitability of shop owners and the e-commerce apps themselves.

However, as complex AI-based retrieval systems, search components are prone to bugs. Search components take multiple factors into account to retrieve products or shops according to user-constructed queries. Prominent ones include semantic similarity between the query and the target, the geographic location of the target, the time of the search, and the corporation's business strategies. To enhance user experience, user preferences and click-through history are also considered, which adds to the complexity. Each step in this complicated retrieval process may introduce bugs, including ones incurring missed recalls.

Missed recalls are estimated to be the second-largest reason for problematic search results in the second quarter of 2023. Figure 1 demonstrates a missed recall reported by the shop owner. For privacy concerns, the information presented is desensitized. As presented in Figure 1, the shop owner initiates two searches targeting his own shop. The target shop can be retrieved when he uses the shop's full name as the query. But it cannot get recalled when he searches for *Chen's*.

Currently, missed recalls are mainly discovered by feedback from shop owners and confirmed heavily relying on manual efforts. To confirm a missed recall in any e-commerce app typically takes four

[1] https://media-meituan.todayir.com/20240322165440176535070 0_tc.pdf



(a) The search result of query "Chen's hardware".

(b) The search result of query "Chen's".

(c) The user reported ticket describing a missed recall of "Chen's"

Figure 1: An example of missed recall

significant steps. Engineers first examine whether the query inducing a missed recall conveys search intention towards the target shop. Then, engineers make sure the location of the search is not too far away from the location of the target shop, as shops far away possess lower priority during retrieval. Next, engineers examine the time of the search. If the target shop is searched out of its opening time, it takes lower priority during retrieval. Finally, engineers consult business operators to check whether the target shop has violated some business policies. If so, the factor of violation punishment may explain this missed recall. After several rounds of communication involving engineers, business operators and the shop owner, the missed recall is confirmed. *Chen's* coveys search intention towards the target shop *Chen's hardware* and *Chen's* is aligned with how general users search on e-commerce apps [2, 3].The shop owner searches for his shop exactly in that shop and during the opening time. This target shop has no record of policy violation. The missed recall presented is confirmed not a false positive.

However, this back-and-forth communication in confirming usually takes days, leaving missed recalls continuing to compromise shop owners' satisfaction. Due to dissatisfaction, the shop owner files a complaint as presented in Figure 1. The long confirming process substantiates the challenging nature of testing towards missed recalls. Subjectivity of search intention expression presents challenges for test case generation, for test cases are user queries conveying search intentions. Diverse and complicated confirming standards can not be easily automated, which adds to the absence of an oracle, as those standards involve manually trading off features of the concerning search component, business strategies and shop status.

3 AN EMPIRICAL STUDY ON HISTORICAL MISSED RECALLS

Given the challenging nature of testing towards missed recalls due to the subjective test cases and absence of an oracle, we first conduct an empirical study based on historical missed recalls. This empirical study provides insights on how general users discover missed recalls and hence inspire the method design.

Specifically, we took user-reported missed recalls handled during January 2023 to June 2023 at Meituan as the study object. There are around 100 entries in total[2] . We qualitatively analyze them by semantic coding [53]. Two authors independently coded all user-reported missed recalls. New codes are created until no new information emerges. After the coding process, the two authors solved differences by discussing and consulting corresponding engineers from the corporation.

We found that the standards used in confirming missed recalls are complicated, detailed, and somewhat ambiguous, which can hardly be automated, proving the challenge of lacking an oracle. Moreover, users typically provide multiple examples, *i.e.* queries for the same target shop, and compare the search results when reporting missed recalls. This finding also motivates us to consider providing an oracle by an metamorphic relation. Here we present the complicated standards in confirming and how users manage to identify missed recalls despite the complicated standards. For privacy concerns, the shop and query information presented is desensitized.

3.1 Ambiguous and Complicated Standards

Both subjective and objective factors are considered to confirm a missed recall. More importantly, the factors considered vary according to different missed recalls. Here we summarize several significant factors.

- **Reasonableness of the query:** As the search component is designed for human usage, confirmed missed recalls can only be induced by *human-perceively reasonable* queries. In practice, a query should align with how general users express shopping needs and convey explicit intention towards the target shop. For example, if a shop named *lovely pets* located 100 meters east of

[2]We purposely omit the actual number for protection of the operational status of the corporation.

Becker street post office can not be retrieved by the query *100 meters east of Becker street post office*, it is not a missed recall. General users usually do not express their shopping needs towards a pet store in that fashion. However, how shopping intentions are expressed involves the subjectivity of human nature, which can hardly be encoded into definite rules.

- **The geography:** Distance affects search results. Shops far away have lower priority during retrieval at the *M-App*. Hence, long distances may cause false positives of missed recall.
- **Operating status:** Usually, only open shops will be retrieved by most e-commerce apps. If a certain shop is closed when a user initiates a search towards it in the *M-App*, it is normal that it can not be retrieved.
- **Ordering strategies:** To enhance user experience, e-commerce apps incorporate multiple ordering strategies, affecting the search results. For example, if a user searches for *hotpot*, those shops offering discounts due to sales promotion may be prioritized during retrieval. Given the fixed number of shops exposed to users, a false impression of missed recall may be presented. To give another example, administrative punishment, lowering the priority in retrieval, works similarly.
- **Others:** User preferences and click-through history of the user affect search results as well. Those factors may also create false impressions of missed recalls.

The above findings demonstrate the complication of identifying missed recall and further prove the oracle problem faced by testing towards missed recalls automatically and proactively. However, despite the complicated standards, general users manage to report missed recalls, with limited knowledge about the operational strategy of the corporation. Hence, we present how general users manage to identify missed recalls and use these findings to inspire our method design.

3.2 Multiple Queries for the Same Shop

On reporting missed recalls, users typically compare the search results of multiple queries towards the same target shop. Accounting for all confirmed user-reported missed recalls (around 80[3]), about 3 queries are constructed for the same shop averagely. Without much knowledge about the confirmation standards used in the corporation, users report missed recalls only when those queries return inconsistent search results. For example, 3 queries are constructed for one target shop, and two queries can recall this shop but one can not. Then, this user reports a missed recall.

After analyzing all queries involved in the user-reported missed recalls we study (over 200 queries [3]), we obtain 2 observations. We observe user queries are diverse and prone to the personal style of expression, which rules can hardly mimic, but what users consider when constructing queries remains stable.

To illustrate the subjectivity and diversity of user queries, we demonstrate the following three types of queries as examples.

- **Equivalent queries.** In concept, the queries are basically the equivalents of the shop name. For example, *hardware Chen's* is an equivalent query for the shop *Chen's hardware*.
- **Including queries.** Including queries conceptually include the target shop. For example, *Indian Restaurant* is an including query for the shop *Ali's curry house*.
- **Included queries.** Included queries are conceptually included by the target shop. For example, *spicy hotpots* is an included query for a restaurant serving spicy and non-spicy hotpots.

Unlike user queries, diverse and subjective, what users consider when constructing queries remains relatively stable. Here we present prominent information types users consider when constructing queries.

- **Shop name.** Users usually take the full name of the shop, characters from the full name of the shop and initials of the shop as queries. For example, given a target shop *Ma's burgers*, users may also search for *Ma's* and *hamburgers*.
- **Products/ services the shop offers.** Users also search for a certain shop by the products or services it offers, like *hotpot*, *haircut* and *pedicures*.
- **Geographic location of the shop.** Users may avoid typing the shop's exact name due to cognitive workload. So they search vaguely by geographic location and construct queries like *hotpot People's square* and *dumplings nearby*.

In summary, findings from the reported missed recalls substantiate the challenges of testing targeting missed recalls and user practices also hint at solutions. Constructing multiple queries towards the same shop and comparing the search results inspires us to consider metamorphic testing to overcome the oracle problem. Subjectivity of human nature leads to diverse queries, which inspires us not to rely on pure rules, but to incorporate subjective factors of humans in test case generation. As users typically consider shop names, services and products the shop provides and the location of the shop when constructing queries, we can use shop names and shop types as input data for test case generation. For shop names and shop types include the three types of information.

4 MRDETECTOR APPROACH

This section presents the technical design of our proposed approach mrDetector. It is an automatic testing approach targeting missed recalls in the search results. Inspired by the findings from section 3, we leverage the ability of LLM to conduct test case generation, which solves the challenge of test cases in line with how general users express their shopping needs. Inspired by the finding that users identify missed recalls by constructing multiple queries for one target shop and comparing the search results, we provide an oracle by a metamorphic relation, which solves the challenge of lacking oracles.

The overall workflow of mrDetector is illustrated in figure 2. mrDetector includes three steps: 1) LLM-based test case generation, 2) test case validation, and 3) missed recall detection.

In the LLM-based test case generation step, mrDetector generates user queries aligning with what general user search with on e-commerce apps, as test cases. In particular, a customized CoT prompt, which mimics how general users construct queries, is used. However, due to the inherent hallucinations in LLMs, test cases not inline with how general users search on e-commerce apps are inevitable. In the test case validation step, the LLM is called again to rethink on and judge the reasonableness of the generated test

[3]We purposely omit the actual number for protection of the operational status of the corporation.

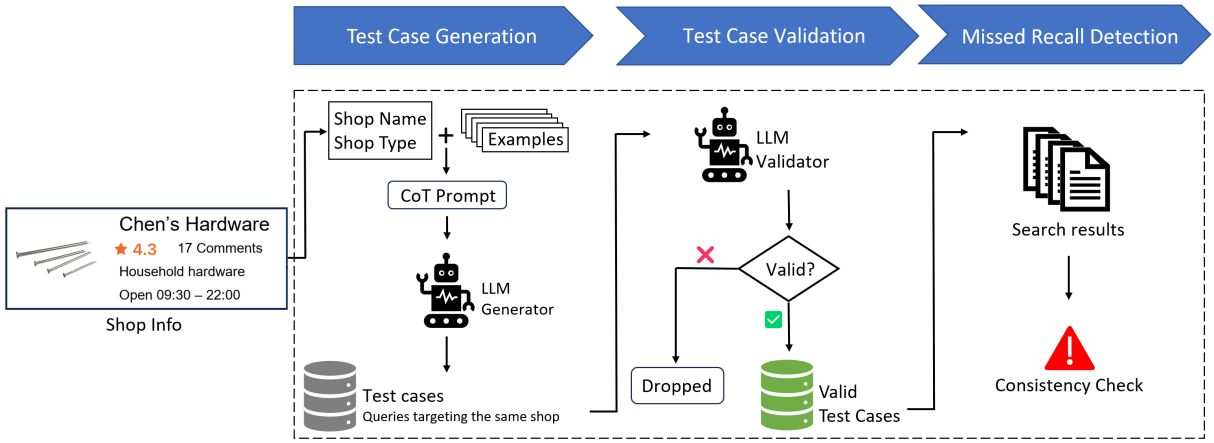


Figure 2: Overview of mrDetector

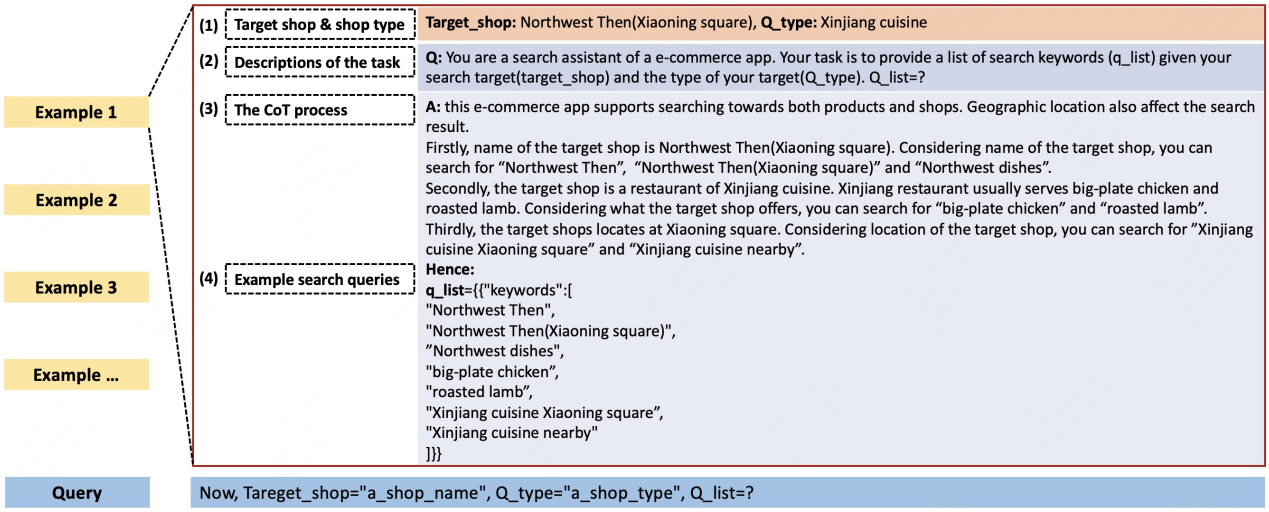


Figure 3: A simplified demonstration of the prompt we use to generate test cases

cases. This step can identify unreasonable test cases that traditional techniques like semantical similarity can not identify and hence reduce false positives. In the missed recall detection step, missed recalls are detected by violation of a metamorphic relation.

4.1 Test Case Generation

In this step we generate test cases/queries automatically by the LLM. As we use a metamorphic relation to provide test oracle, our test cases (essentially user queries) must: (1) be real enough, approximating to user-constructed ones; (2) support the metamorphic relation. To generate real test cases, we leverage the ability of the LLM instead of traditional NLP techniques. LLMs are trained with a large corpus of natural languages, implicitly conveying how humans express their shopping needs and interact with search components. It is reasonable to assume that given the information needed, the LLM could generate queries that approximate what users search with on the search component. After trading off the costs and base abilities, we choose GPT-3.5 turbo for query generation. To support the metamorphic relation, we generate a group of test cases/queries for one shop instead of a single test case/query. Those queries all target the same shop and express search intention towards it.

We tailor the prompt to mimic how general users construct queries during their search behavior. According to what information general users consider when constructing queries, we take shop names and shop types as the input. The shop names also include information about where the shop is located. To leverage the ability of LLM to generate queries close to what general users construct during searching, we use a series of techniques, including in-context learning[42], CoT[66], question-answering examples [9] and human in the loop[22].

We first break the query generation process of general users (illustrated in section 3.2) into three steps (shop name based generation, services/products based generation and location based generation) according to the information types they consider. Then we accordingly implement the three steps into the prompt with the CoT technique. Next we use the in-context learning technique to provide domain knowledge about our specific task and include multiple examples in the prompt. As LLMs benefit from examples in a question-answer format in the prompt [9], all examples are

provided in the question-answer format. To ensure the quality of all the examples in the prompt, we randomly sample from real user queries of Meituan and manually revise them to achieve consistency in our scenario. We also use human evaluation to substitute all examples that general users think "*not real enough*" due to revision. Finally, we use the human-in-the-loop technique and iteratively improve our prompt based on the human feedback on the quality of the queries generated. We stop the iteration process until no unreal ones are found in randomly sampled 50 generated queries. Our final prompt consists of four parts: (1) target shops and shop types; (2) descriptions of the task; (3) the CoT process of query generation; (4) example search queries. The prompt is written in Chinese, as the input data are also in Chinese. Figure 3 provides a simplified demonstration of our prompt. For privacy concerns, the shop information and example queries presented are desensitized.

4.2 Test Case Validation

Due to hallucination issues, LLMs may generate unreliable content despite the customized CoT prompt to ensure generation quality. For example, the query *a barber* is generated for the target shop, a hairdressing salon. Another example is *supermarket near A* for the target shop *A supermarket*, which confuses A as a location.

Those test cases are human-perceively unreasonable but can not be detected by traditional techniques. For example, the *a barber* scenarios would achieve high semantical similarity scores and the *supermarket near A* scenarios would achieve high textual matching scores.

Hence, we leverage a rethink scheme [38] to reduce false positives induced by unreasonable test cases in the test case validation step. Specifically, given the name and type of the target shop and the generated test case, we call the LLM again to re-think the generated test cases and drop ones that do not align with how general users express their shopping needs. The rationale is that multiple trials with the LLM decrease the probability of unreliable responses incurred by hallucination. Only test cases validated as **resonanble** are kept for the next step. The prompt used for test case validation follows the few-shot strategy. Multiple examples, both reasonable test cases to be kept and unreasonable test cases to be dropped, are included in the prompt.

4.3 Missed Recall Detection

In this step, we determine whether a certain test case incurs a missed recall by consistency checks. We propose the metamorphic relation that test cases generated for one target shop should present consistent search results. Violations of this metamorphic relation indicate missed recalls. Hence, the test oracle is formally expressed below.

Given test cases $X_1, X_2, \ldots, X_n$ for a target shop t, the function $R(X)$ represent search results recalled by X . Formally, for any X_n, we define y_n as follows:

$$y_n = \begin{cases} \text{True} & \text{if } t \in R(X_n), \\ \text{False} & \text{if } t \notin R(X_n). \end{cases} \tag{1}$$

The oracle suggests a potential missed-recall when, for any pair i, j where $1 \leq i, j \leq n$, we have $y_i \neq y_j$.

We report test cases that do not recall the target shops as those incurring missed recalls only when other test cases with the same target shop can recall it. To prevent false positives, we are particularly cautious about the situation that all test cases towards a specific shop can not recall the target shop. For example, if a shop decides to cease its business cooperation with an e-commerce app, whatever user queries you try on the search component of that app can not recall this shop. Those situations should not report missed recalls.

In implementation, an internal test API that provides the same functionalities to the search component is used to get all search results. As shown in section 3.1, apart from the query, many other factors also affect the search results of search components in e-commerce apps. To prevent false positives, we use the same user account to ensure the same user profiling and change the location of searches to the accurate geographic locations of the target shops to prevent the explainability of long distances. We also conduct searches during the time slot of 10 am. to 9 pm., when most shops are open, which prevents the explainability of shop status.

5 EVALUATION

In this section, we evaluate the performance of mrDetector by answering the following research questions (RQs):

- **RQ1**: How do different LLMs affect the performance of mrDetector?
- **RQ2**: How do different strategies of prompt engineering affect the performance of mrDetector?
- **RQ3**: How does the LLM validation step affect the performance of mrDetector?
- **RQ4**: How does mrDetector perform in detecting real online missed recalls?

RQ1, RQ2 and RQ3 target the effect of each part in mrDetector, *i.e.*, the LLM used, the prompt for test case generation and the validation step, on its performance. RQ4 treats mrDetector as a whole, and aims to illustrate its handiness in detecting real online missed recalls of a prevalent e-commerce app in China.

5.1 Experimental Setup

Datasets. We use two datasets to evaluate the performance of mrDetector.

- **Dataset A**: Dataset A is a manually constructed open-access dataset consisting of 600 entries of shops. The shop entries are collected via the Baidu Map API[4], which returns nearby shops given a geographic location. We randomly sampled these shops from all shops in Beijing and Shanghai via the API in November 2023. Those 600 entries cover most daily life services, including hairdressing, skin care, nail polishment and catering. We believe such diverse shop types could unveil the ability of mrDetector to detect general missed recalls, avoiding the possibility that mrDetector overfits on a certain shop type.
- **Dataset B**: 600 entries of shops randomly sampled from the business partner table of Meituan. This table records all shops that have established business cooperation with Meituan. The

[4] https://api.map.baidu.com/place/v2/search

Table 1: How to present a shop in our datasets and the purpose of each attribute.

Shop name	Shop type	City	Latitude	Longitude
Old Flavor Hotpot	Beijing hotpot	Beijing	116.3E	40.5N
Input data for test case generation		**Detailed location to reduce test false positives**		

shop types are in line with that of Dataset A. Dataset B is a dataset of real business data.

Dataset A and Dataset B employ the same data format and are all in Chinese. As shown in Table 1, we demonstrate how each shop is presented in each entry with a mock shop.

Metrics. We use Reported Cases ($N_{reported}$), Confirmed Cases ($N_{Confirmed}$), False Positive Ratio (R_{fp}), and Test Case Efficiency (E_{tc}) to measure the performance of mrDetector.

$N_{Reported}$ and $N_{Confirmed}$ describe the **benefit** side of mrDetector. $N_{reported}$ measures how many missed recalls are reported by mrDetector. Higher $N_{reported}$ indicates stronger ability to discover missed recalls. $N_{Confirmed}$ measures how many reported missed recalls are confirmed by human. Higher $N_{Confirmed}$ indicates higher accuracy of mrDetector. We present $N_{Reported}$ and $N_{Confirmed}$ both entry-wise and shop-wise. R_{fp} and E_{tc} describe the **cost** side of mrDetector, and are calculated as:

$$R_{fp} = \frac{N_{Reported} - N_{Confirmed}}{N_{Reported}} \tag{2}$$

$$E_{tc} = \frac{N_{Total}}{N_{confirmed}} \tag{3}$$

where N_{Total} refers to the total number of test cases/queries during one round of test. Lower R_{fp} indicates less human efforts incurred by false positives in confirming. E_{tc} measures the average number of test cases needed to find a confirmed missed recall. Lower E_{tc} hints a smaller number of test cases needed to find the same amount of missed recalls and hence a higher test efficiency. R_{fp} and E_{tc} are reported entry-wise.

Experimental environment. We implement mrDetector with 612 lines of Python codes. All experiments are conducted on an Ubuntu 20.04 server with Intel (R) Xeon (R) Platinum 8175M CPU (2.50GHz), 48GB RAM and two NVIDIA GeForce RTX 4090 GPUs (210MHz).

Manual Confirmation Procedure. Manual confirmation of reported missed recalls requires examining the reasonableness of test cases, which is prone to the subjectivity of the conductor. During confirmation, two authors first independently confirm reported missed recalls. Then, they solve differences by discussing and consulting a third party. We believe such a cross-validating procedure curbs the effects of human subjectivity on confirmation results.

5.2 Performance with Different LLMs (RQ1)

Baselines. mrDetector use GPT-3.5 turbo, which is estimated to have over 100B parameters [62], to conduct test case generation. Here we compare the performances of mrDetector with LLMs of different parameter sizes to unveil how the selection of the LLM affects the performance. Besides mrDetector, we implement three baseline methods with three LLMs of different parameter sizes.

- **GPT-Neo Version**: mrDetector implemented with GPT-Neo. GPT-Neo [8] is a pre-trained language model following the transformer architecture. According to the official document, it has around 2.6B parameters.
- **ChatGLM2 Version**: mrDetector implemented with ChatGLM2. ChatGLM2 is an open bilingual language model based on General Language Model framework [18]. It leverages both the Chinese and English languages and has approximately 6B parameters.
- **Qwen Version**: mrDetector implemented with Qwen 14-B. It [5] is a 14B-parameter version of the large language model series, Qwen (abbr. *Tongyi Qianwen*), proposed by *Alibaba Cloud*.

Results. We answer RQ1 with Dataset A. As demonstrated in Table 2, comprehensively mrDetector performs best with GPT-3.5 turbo, which has the largest parameter size. Specifically, on the cost side mrDetector only reports one false positive and only needs 80.95 test cases to discover a confirmed missed recall with GPT-3.5 turbo. This achieves the best among all LLMs concerned. On the benefit side, mrDetector built on GPT-3.5 turbo shows a comparable ability with that built on Qwen, which performs best with benefit metrics.

According to the results in Table 2, a conclusion could be loosely drawn that larger parameters benefit the performance of mrDetector. Analysis with false positives also confirms this conclusion. Test cases not in line with what general users search with on e-commerce apps are the most significant reason for incurring false positives. Chat-NEO, which has the minimal-sized parameters, often outputs random characters or emojis that hardly convey search intentions toward the target shops. Chat-GLM2 and Qwen do not output random characters but sometimes fail to catch the subjective way general users express shopping needs via user queries. For example, in the Chinese language context, a *seafood joint* usually refers to where sells raw seafood. While a *seafood restaurant* instead refers to a restaurant that sells seafood dishes. So general users seldom use *seafood joint* to search for a seafood restaurant on e-commerce apps. GPT-3.5 turbo, which has the maximal-size parameters, only generates one test case that is not real enough. Analysis with false positives may also partially explain why mrDetector build on Qwen performs best with benefit metrics. Qwen includes a larger ratio of Chinese corpus in the training data. Hence, it may enjoy advantages in dealing with Chinese input data.

> **Summary:** A larger parameter size generally facilitates the ability to generate valid test cases of mrDetector. Comprehensively mrDetector built on GPT-3.5 turbo achieves the best performance. It reports 46 confirmed missed recalls with only one false positive.

5.3 Performance with Different Prompt Engineering Strategies (RQ2)

Baselines. mrDetector uses a customized CoT prompt to generate test cases that both align with users' search behavior and support our metamorphic relation. Here we investigate how different prompt engineering strategies affect the performance of mrDetector. Besides mrDetector, we implement two baseline methods with two different prompt engineering strategies.

- **Few-shot Version**: mrDetector implemented with a standard few-shot prompt and GPT-3.5 turbo. The standard few-shot prompt

Table 2: Reported Cases ($N_{reported}$), Confirmed Cases ($N_{Confirmed}$), False Positive Ratio (R_{fp}), and Test Case Efficiency (E_{tc}) comparison among mrDetector versions implemented with LLMs of different parameter sizes.

Versions		Benefit metics		Cost metrics	
LLMs	Parameters	$N_{reported}\uparrow$	$N_{Confirmed}\uparrow$	$R_{fp}\downarrow$	$E_{tc}\downarrow$
GPT-Neo	2.6B	35 entries/ 35 shops	6 entries/6 shops	29/35=0.829	2607/6=434.500
Chat-GLM2	6B	54 entries/ 44 shops	32 entries/26 shops	22/54=0.407	3803/32=118.844
Qwen	14B	**64 entries/ 48 shops**	**54 entries/40 shops**	10/54=0.156	4375/54=81.019
GPT-3.5 turbo	over 100B	47 entries/ 33 shops	46 entries/33 shops	**1/47=0.021**	**3724/46=80.95**

includes all examples in the customized CoT prompt, but no hints about the *steps* of test case generation are provided.

- **Zero-shot Version**: mrDetector implemented with a standard zero-shot prompt and GPT-3.5 turbo. Only textual descriptions of the generation task are included in the zero-shot prompt. No examples and hints about the *steps* of test case generation are provided.

Results. We answer RQ2 with Dataset A. As demonstrated in Table 3, comprehensively mrDetector performs best with the customized CoT prompt. mrDetector implemented with a standard few-shot prompt generates far more test cases than that implemented with the zero-shot prompt and customized CoT prompt. Hence, it is reasonable that it reports the most missed recalls. Judging from the benefit side, the few-shot version discovers around three times more missed recalls than the customized CoT prompt version. However, judging from the cost side, it reports over 30 times more false positives than the customized CoT prompt version. Trading off costs and benefits, comprehensively, the customized CoT prompt achieves the best performance with its significantly low R_{fp}.

We develop two assumptions based on the above observation and analysis with test cases generated. Firstly, examples in the prompt may increase the number of test cases generated. Examples are materials that the LLM can learn from and mimic, hence may inspire more generation results. This partially explains why the few-shot version generates the most test cases. Secondly, the steps stating how a generation task should be solved and contained in the customized CoT prompt may guide how the LLM *thinks*. So, how the LLM generates test cases may approximate how general users construct queries, and hence less test cases, especially unreal test cases, are generated. This assumption partially explains why the customized CoT prompt version archives the lowest R_{fp}, six times lower than the second lowest.

> **Summary:** Comprehensively mrDetector built on the customized CoT prompt performs best with its lowest R_{fp}, six times lower than the second lowest. Firstly, examples included in the prompt may increase the number of test cases generated. Secondly, the steps stating how a generation task should be solved and contained in the customized CoT prompt may guide how the LLM *thinks* and reduce false positives.

5.4 Performance with/without the LLM Validation (RQ3)

Baseline. To reduce the impact of hallucination issues, mrDetector leverage a rethink scheme and call the LLM again to validate the generated test cases. To illustrate how effective this LLM validation step is, we implement the **Without LLM Validation Version** of mrDetector as the baseline. Except for omitting the LLM validation step, no further changes are made.

Results. We answer RQ3 with Dataset A. As shown in Table 4, the LLM validation step significantly reduces the false positive ratio (3.28 times). ALL false positives the validation step prevents are due to edge cases not aligning with how general users search on e-commerce apps. For example, *Shanghai [space] restaurant* is generated for a restaurant serving Shanghai Cuisine. However, in the Chinese context, this test case is generally interpreted as *restaurants in Shanghai*, instead of *Shanghai Cuisine Restaurants*. This edge case, which can not be easily detected by traditional techniques like semantic similarity and textual mating, proves an LLM validation step is necessary.

This validation step with LLM helps reduce the count of missed recalls that were reported and confirmed due to our cautious approach in the experiment. When the LLM cannot provide a clear decision for test cases, we consider them as failing the LLM check step. This experimental approach reduces the number of test cases and consequently lowers the count of reported missed recalls.

> **Summary:** The LLM validation step brings down the false positive ratio 3.28 times, only compromising 35 percent of confirmed missed recalls. It can prevent false positives incurred by edge cases not aligning with how general users search on e-commerce apps and can not easily detected by traditional techniques.

5.5 Performance in Discovering Real Online Missed Recalls (RQ4)

To show how well mrDetector works in real industries, we use Dataset B, which contains actual industry data. This data, handled by different departments, is often untidy. While doing well with open data can demonstrate algorithmic strengths, it doesn't guarantee success in real industrial scenarios, where results are affected by both algorithms and other factors.

Results. In total, 6396 test cases are generated by mrDetector. 118 entries of missed recalls (involving 91 shops) are reported. After human confirmation, 101 entries (involving 76 shops) are confirmed. The R_{fp} is 0.144, which means false positives take 14 percent of human efforts during confirming. The E_{tc} reaches 63.327. This demonstrates that on average 63 test cases are needed to find a confirmed missed recall.

Revisiting the results of RQ1 to RQ3, we find mrDetector generates more test cases and finds more missed recalls with industrial data. One possible explanation is that the customized CoT prompt is inspired by the historical missed recalls, which are based on real

Table 3: Reported Cases ($N_{reported}$), Confirmed Cases ($N_{Confirmed}$), False Positive Ratio (R_{fp}), and Test Case Efficiency (E_{tc}) comparison among mrDetector versions implemented with different prompt engineering strategies.

Versions		Benefit metics		Cost metrics	
Prompts	info.	$N_{reported}\uparrow$	$N_{Confirmed}\uparrow$	$R_{fp}\downarrow$	$E_{tc}\downarrow$
Few-shot	examples	**156 entries/ 98 shops**	**121 entries/76 shops**	35/156=0.223	**5700/121=47.107**
Zero-shot	-	46 entries/ 42 shops	39 entries/37 shops	7/46=0.152	4622/39=118.513
Customized CoT	Examples, steps	47 entries/ 33 shops	46 entries/33 shops	**1/47=0.021**	3724/46=80.95

Table 4: Reported Cases ($N_{reported}$), Confirmed Cases ($N_{Confirmed}$), False Positive Ratio (R_{fp}), and Test Case Efficiency (E_{tc}) comparison among mrDetector versions implemented with/without the LLM validation step.

Versions	Benefit metics		Cost metrics	
	$N_{reported}\uparrow$	$N_{Confirmed}\uparrow$	$R_{fp}\downarrow$	$E_{tc}\downarrow$
without LLM validation	**78 entries/ 45 shops**	**71 entries/42 shops**	7/78=0.090	**3724/71=52.451**
with LLM Validation	47 entries/ 33 shops	46 entries/33 shops	**1/47=0.021**	3724/46=80.95

industrial data. This may benefit the performance of mrDetector in a real industrial setting.

We also find that mrDetector reports more false positives with real industrial data. During confirmation, we find those false positives are incurred not only by unreal test cases (the same as in RQ1, RQ2, and RQ3) but also by the quality of the input data. Although not common, it exists that the shop type provided is reasonable but too broad to describe a shop from the subjective aspects of humans. For example, a shop curing nail fungus often is not considered a *nail polish center* by general users, although it does deal with nails.

Effective as mrDetector is in detecting online missed recalls, according to engineers in charge, it can further help reduce future missed recalls if it reports missed recalls that are **reproducible** on the **phone**. To prepare fixes, the first step for engineers is to reproduce those missed recalls on the phone and analyze their impact on the user experience. Hence, although reproducibility does not compromise the effectiveness, it increases the handiness of mrDetector in a real industrial setting. To this end, we randomly sample 15 confirmed missed recalls and manually reproduce them on a mobile phone. Specifically, we search for the same test cases/user queries on the *M-App* installed on an iPhone 13, and check whether the same missed recalls appear. During the searches, we roughly change the geographic location by filling in the nearest landmark buildings to the target shops. Over half (8/15) of missed recalls can be reproduced. After analyzing those that can not be reproduced, we find the most prominent reason lies in the outdatedness of the target shop information. Some shops no longer exist but they haven't been erased from the business partner table. So, they are taken as target shops during the test. This explains 4 out of 7 missed recalls that can not be reproduced. This reproduction experiment further illustrates the handiness of mrDetector, besides its effectiveness in detecting missed recalls.

Summary: mrDetector finds 118 missed recalls with real industrial data. Only 17 are confirmed to be false positives. Over half of the missed recalls confirmed are estimated to be reproducible. The customized CoT prompt of mrDetector is inspired by the historical missed recalls, which are based on real industrial data. This may benefit the performance of mrDetector in a real industrial setting.

6 CASE STUDIES

Due to a large number of missed recalls found and limited human resources, we randomly sampled 8 representative missed recalls to discuss root causes with corresponding engineers. They accept all 8 missed recalls and summarize two major reasons causing missed recalls[5]. After discussion with corresponding engineers, mrDetector will be launched to scan for missed recalls for Meituan on a regular basis.

6.1 Segmenting Related Missed Recalls

General users construct natural language queries to express their search intentions. The search component first segments the query and decides which part conveys the main search intention. For some user queries like *Fresh**de SPA*, where the two parts both carry important information about the user's intention, choosing a *main part* might be tricky. For example, if the search component takes *Fresh**de* as the part mainly carrying user search intentions, shops whose title contains the characters *Fresh**de* could be recalled. In this case, as shops like *Fresh**de Laundry* and *Fresh**de Massage* exist nearby, the shop *Fresh**de Healthy SPA* may be neglected. Hence, a missed recall occurs.

6.2 Landmark Related Missed Recalls

Some shop names contain locations. When a user wants to search for that shop on an e-commerce app, often a phrase referring to a location will be contained in the user query. When the search component gets this query, it must decide whether this location phrase specifies the location of the target shop or only states the name of the target shop. This judgment may lead to missed recalls. For example, there's a shop named *F**'s Seafood Barbecue (Fangbang)*. When searching with *Barbecue Fangbang*, the search component may loosely translate this query as *find barbecue shops located at Fangbang*. As so many barbecue shops exist at Fangbang and only several of them can be presented to users, it is possible, although rare, that the shop *F**'s Seafood Barbecue (Fangbang)* didn't get recalled. Hence, a missed recall occurs.

[5]Due to privacy concerns, some characters in the shop name are replaced with *s.

7 THREATS TO VALIDITY

As mrDetector relies on GPT-3.5 turbo to generate test cases, stability of the service and hallucination issues may hinder the effectiveness of mrDetector. Firstly, incidents like network thrashing, API errors, and time-out errors can cause API calls to fail. To cope with this, we leverage the retry schema. If no content is returned after 30 seconds, we terminate the thread, wait a random period, and retry three times. Secondly, LLMs can not guarantee the reliability of the generated content due to hallucination. Thus, unreal test cases can be generated by mrDetector and introduce false positives. To cope with this, we customize a CoT prompt that mimics how general users construct user queries, leverage a *rethink* step, and set the temperature to zero in implementation.

Outside of the approach itself, the quality of input data affects the performance of mrDetector. Outdated input data as well as incorrectness in input data incur false positives. For example, if shops that no longer exist are taken as target shops, only false positives of missed recalls can be found.

8 RELATED WORK

Testing of Search Components. For user experience, search components are expected only to recall all relevant search results. Hence, precision and recall [27] have always been valued. However, relevance is considered a rather subjective concept [48]. As a result, search components were usually tested with human judgments [24, 52] in the 2000s. For example, a good ranking strategy is expected to generate rankings similar to human-generated ones [57]. With the development of deep learning techniques, search components also leverage deep neuron networks to enhance their performances [20, 30, 34, 45, 64]. The explainability issues [4] of deep neuron networks contribute to the oracle problem and motivate the introduction of metamorphic testing. [67] summarizes five groups of metamorphic relations and applies them to general-purpose search engines. Metamorphic testing can also be used in domain-specific search components like academic search facilities [13], e-commerce search functionalities [44], and code search engines [17]. To conduct metamorphic testing, the key step is to construct a reliable metamorphic relation, which serves as the oracle during testing. [49] provides an automatic method for generating metamorphic relations, which helps to alleviate the labor of constructing metamorphic relations.

LLM and Testing. Generally, LLMs are pre-trained language models following the Transformer architecture [56]. They usually feature large-scale training corpus and massive parameters. To leverage those pre-trained LLMs effectively, a group of researchers fine-tuned the models with downstream datasets [16] before using them. LLMs have far more parameters than ordinary pre-trained language models, drastically increasing fine-tuning costs. To cope with this challenge, a series of cost-efficient strategies for fine-tuning are proposed, such as Lora [29] and prefix tuning [35]. However, fine-tuning LLMs needs a large quantity of downstream data, which may not be accessible. Hence, in-context learning is an alternative [59]. Empirical studies are also conducted towards constructing more efficient prompts during in-context learning, such as [21]. Due to their superior performances, LLMs are applied to multiple fields of research [1, 6, 7, 19, 31, 47, 60], including tasks of software engineering [23].

In testing, LLMs can be leveraged in test case generation. TitanFuzz [15] uses a generative LLM to produce initial seed programs from target APIs and conduct mutation with them by LLMs to generate test cases for deep learning libraries. As TitanFuzz tends to generate ordinary programs, but edge cases generally induce more bugs, FuzzGPT [15] is proposed to learn from historical bug-triggering programs and generate similar test cases by LLMs. Bug reports are also effective resources for LLMs to generate bug-triggering programs [32]. Apart from generating test cases directly, LLMs can also guide the generating process to escape the coverage plateaus [33].

Metamorphic Testing. Metamorphic testing [12, 37] is typically applied to untestable situations where a direct testing oracle is challenging to acquire. The core idea is to detect violations of metamorphic relations among multiple outputs [11, 50]. Many practices of metamorphic testing can be witnessed in the domain of software engineering [10, 36, 41]. Viewed as black boxes, AI-based systems also promote the application of metamorphic testing. It can be used to both classify algorithms [43, 61], and software built upon them [54, 58, 63, 65]. For demonstration, we take machine translation software as an example. Machine translation software typically leverages deep neuron networks to transfer one natural language to another. As the subjectivity of human expression, multiple sentences expressing the same semantics exist. Hence, there is no standard answer for a translation that can be used as the testing oracle. However, it can be observed that normally, sentences conveying different meanings should not have the exact translation, and translation of similar-structured sentences should typically exhibit the same sentence structure [25, 28]. Hence, metamorphic relations could be established.

9 CONCLUSION

In this paper, we present mrDetector, the first testing approach targeting missed recalls of e-commerce search components. mrDetector leverages an LLM, specifically GPT-3.5 turbo and a customized CoT prompt inspired by the historical missed recalls, to mimic how general users construct queries during online shopping and it relies on a metamorphic relation to find missed recalls. Experiments with open data demonstrate its performance advantages to baselines. Experiments with private industrial data show that mrDetector discovers 101 confirmed missed recalls and only reports 17 false positives in the real industrial setting. Over half of the missed recalls which mrDetector discovered can be reproduced.

ACKNOWLEDGMENTS

This work is supported by Meituan and the Natural Science Foundation of Shanghai (Project No. 22ZR1407900). We extend our heartfelt thanks to our colleagues in Meituan, specifically, You, Jingjian, Pingping, Xiaolan, Ying, and Qinling, for their kind help and support in this work. Y. Zhou is the corresponding author.

REFERENCES

[1] Toufique Ahmed, Supriyo Ghosh, Chetan Bansal, Thomas Zimmermann, Xuchao Zhang, and Saravan Rajmohan. 2023. Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models. *arXiv preprint*

arXiv:2301.03797 (2023).

[2] Qingyao Ai, Daniel N Hill, SVN Vishwanathan, and W Bruce Croft. 2019. A zero attention model for personalized product search. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. 379–388. https://doi.org/10.1145/3357384.3357980

[3] Qingyao Ai, Yongfeng Zhang, Keping Bi, Xu Chen, and W Bruce Croft. 2017. Learning a hierarchical embedding model for personalized product search. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 645–654. https://doi.org/10.1145/3077136.3080813

[4] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-López, Daniel Molina, Richard Benjamins, et al. 2020. Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. *Information fusion* 58 (2020), 82–115. https://doi.org/10.1016/j.inffus.2019.12.012

[5] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. *arXiv preprint arXiv:2309.16609* (2023).

[6] Som S Biswas. 2023. Potential use of chat gpt in global warming. *Annals of biomedical engineering* 51, 6 (2023), 1126–1127. https://doi.org/10.1007/s10439-023-03171-8

[7] Som S Biswas. 2023. Role of chat gpt in public health. *Annals of biomedical engineering* 51, 5 (2023), 868–869. https://doi.org/10.1007/s10439-023-03172-7

[8] Sid Black, Gao Leo, Phil Wang, Connor Leahy, and Stella Biderman. 2021. *GPT-Neo: Large Scale Autoregressive Language Modeling with Mesh-Tensorflow.* https://doi.org/10.5281/zenodo.5297715

[9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[10] WK Chan, Shing Chi Cheung, and Karl RPH Leung. 2005. Towards a metamorphic testing methodology for service-oriented software applications. In *Fifth International Conference on Quality Software (QSIC'05)*. IEEE, 470–476. https://doi.org/10.1109/qsic.2005.67

[11] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, TH Tse, and Zhi Quan Zhou. 2018. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys (CSUR)* 51, 1 (2018), 1–27.

[12] Tsong Yueh Chen, Tsun Him Tse, and Z. Quan Zhou. 2003. Fault-based testing without the need of oracles. *Information and Software Technology* 45, 1 (2003), 1–9. https://doi.org/10.1016/s0950-5849(02)00129-5

[13] Stevão Alves de Andrade, Ítalo Santos, Claudinei Brito Junior, Misael Júnior, Simone RS de Souza, and Márcio E Delamaro. 2019. On applying metamorphic testing: an empirical study on academic search engines. In *2019 IEEE/ACM 4th International Workshop on Metamorphic Testing (MET)*. IEEE, 9–16. https://doi.org/10.1109/met.2019.00010

[14] Jon Degenhardt, Surya Kallumadi, Utkarsh Porwal, and Andrew Trotman. 2021. Report on the SIGIR 2019 Workshop on eCommerce (ECOM19). In *ACM SIGIR Forum*, Vol. 53. ACM New York, NY, USA, 11–19. https://doi.org/10.1145/3458553.3458555

[15] Yinlin Deng, Chunqiu Steven Xia, Chenyuan Yang, Shizhuo Dylan Zhang, Shujing Yang, and Lingming Zhang. 2023. Large language models are edge-case fuzzers: Testing deep learning libraries via fuzzgpt. *arXiv preprint arXiv:2304.02014* (2023).

[16] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).

[17] Zuohua Ding, Qingfen Zhang, and Mingyue Jiang. 2020. Metamorphic Testing of Code Search Engines. In *2020 International Symposium on Theoretical Aspects of Software Engineering (TASE)*. IEEE, 177–184. https://doi.org/10.1109/tase49443.2020.00032

[18] Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, and Jie Tang. 2022. GLM: General Language Model Pretraining with Autoregressive Blank Infilling. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 320–335. https://doi.org/10.18653/v1/2022.acl-long.26

[19] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated repair of programs from large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1469–1481. https://doi.org/10.1109/icse48619.2023.00128

[20] Debasis Ganguly, Dwaipayan Roy, Mandar Mitra, and Gareth JF Jones. 2015. Word embedding based generalized language model for information retrieval. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. 795–798. https://doi.org/10.1145/2766462.2767780

[21] Shuzheng Gao, Xin-Cheng Wen, Cuiyun Gao, Wenxuan Wang, and Michael R Lyu. 2023. Constructing Effective In-Context Demonstration for Code Intelligence Tasks: An Empirical Study. *arXiv preprint arXiv:2304.07575* (2023).

[22] Yingqiang Ge, Wenyue Hua, Jianchao Ji, Juntao Tan, Shuyuan Xu, and Yongfeng Zhang. 2023. Openagi: When llm meets domain experts. *arXiv preprint arXiv:2304.04370* (2023).

[23] Robert L. Glass, Iris Vessey, and Venkataraman Ramesh. 2002. Research in software engineering: an analysis of the literature. *Information and Software technology* 44, 8 (2002), 491–506. https://doi.org/10.1016/s0950-5849(02)00049-6

[24] Michael Gordon and Praveen Pathak. 1999. Finding information on the World Wide Web: the retrieval effectiveness of search engines. *Information processing & management* 35, 2 (1999), 141–180. https://doi.org/10.1016/s0306-4573(98)00041-7

[25] Shashij Gupta, Pinjia He, Clara Meister, and Zhendong Su. 2020. Machine translation testing via pathological invariance. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 863–875. https://doi.org/10.1145/3368089.3409756

[26] Aniko Hannak, Piotr Sapiezynski, Arash Molavi Kakhki, Balachander Krishnamurthy, David Lazer, Alan Mislove, and Christo Wilson. 2013. Measuring personalization of web search. In *Proceedings of the 22nd international conference on World Wide Web*. 527–538. https://doi.org/10.1145/2488388.2488435

[27] David Hawking, Nick Craswell, Peter Bailey, and Kathleen Griffihs. 2001. Measuring search engine quality. *Information retrieval* 4, 1 (2001), 33–59.

[28] Pinjia He, Clara Meister, and Zhendong Su. 2020. Structure-invariant testing for machine translation. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 961–973. https://doi.org/10.1145/3377811.3380339

[29] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685* (2021).

[30] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. 2020. Embedding-based retrieval in facebook search. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2553–2561. https://doi.org/10.1145/3394486.3403305

[31] Wenxiang Jiao, Wenxuan Wang, Jen-tse Huang, Xing Wang, and Zhaopeng Tu. 2023. Is ChatGPT a good translator? A preliminary study. *arXiv preprint arXiv:2301.08745* (2023).

[32] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323. https://doi.org/10.1109/icse48619.2023.00194

[33] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. 2023. CODAMOSA: Escaping coverage plateaus in test generation with pre-trained large language models. In *International conference on software engineering (ICSE)*. https://doi.org/10.1109/icse48619.2023.00085

[34] Sen Li, Fuyu Lv, Taiwei Jin, Guli Lin, Keping Yang, Xiaoyi Zeng, Xiao-Ming Wu, and Qianli Ma. 2021. Embedding-based product retrieval in taobao search. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 3181–3189. https://doi.org/10.1145/3447548.3467101

[35] Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190* (2021).

[36] Christopher Lidbury, Andrei Lascu, Nathan Chong, and Alastair F Donaldson. 2015. Many-core compiler fuzzing. *ACM SIGPLAN Notices* 50, 6 (2015), 65–76. https://doi.org/10.1145/2813885.2737986

[37] Huai Liu, Fei-Ching Kuo, Dave Towey, and Tsong Yueh Chen. 2013. How effectively does metamorphic testing alleviate the oracle problem? *IEEE Transactions on Software Engineering* 40, 1 (2013), 4–22. https://doi.org/10.1109/tse.2013.46

[38] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2023. Make LLM a Testing Expert: Bringing Human-like Interaction to Mobile GUI Testing via Functionality-aware Decisions. *arXiv preprint arXiv:2310.15780* (2023).

[39] Chen Luo, William Headden, Neela Avudaiappan, Haoming Jiang, Tianyu Cao, Qingyu Yin, Yifan Gao, Zheng Li, Rahul Goutam, Haiyang Zhang, et al. 2022. Query attribute recommendation at Amazon Search. In *Proceedings of the 16th ACM Conference on Recommender Systems*. 506–508. https://doi.org/10.1145/3523227.3547395

[40] Michael R Lyu et al. 1996. *Handbook of software reliability engineering*. Vol. 222. IEEE computer society press Los Alamitos.

[41] Muhammad Numair Mansur, Maria Christakis, and Valentin Wüstholz. 2021. Metamorphic testing of Datalog engines. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 639–650. https://doi.org/10.1145/3468264.3468573

[42] Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2022. Rethinking the role of demonstrations: What makes in-context learning work? *arXiv preprint arXiv:2202.12837* (2022).

[43] Christian Murphy, Gail E Kaiser, and Lifeng Hu. 2008. Properties of machine learning applications for use in metamorphic testing. (2008). https://doi.org/10.

7916/D8XK8PFD

[44] Shu Nagai and Tatsuhiro Tsuchiya. 2018. Applying metamorphic testing to e-commerce product search engines. In *2018 IEEE 23rd Pacific Rim International Symposium on Dependable Computing (PRDC)*. IEEE, 183–184. https://doi.org/10.1109/prdc.2018.00030

[45] Priyanka Nigam, Yiwei Song, Vijai Mohan, Vihan Lakshman, Weitian Ding, Ankit Shingavi, Choon Hui Teo, Hao Gu, and Bing Yin. 2019. Semantic product search. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2876–2885.

[46] Will B Payne. 2021. Powering the local review engine at Yelp and Google: intensive and extensive approaches to crowdsourcing spatial data. *Regional Studies* 55, 12 (2021), 1878–1889. https://doi.org/10.1080/00343404.2021.1910229

[47] Hammond Pearce, Benjamin Tan, Baleegh Ahmad, Ramesh Karri, and Brendan Dolan-Gavitt. 2023. Examining zero-shot vulnerability repair with large language models. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2339–2356. https://doi.org/10.1109/sp46215.2023.10179324

[48] Tefko Saracevic. 1975. Relevance: A review of and a framework for the thinking on the notion in information science. *Journal of the American Society for information science* 26, 6 (1975), 321–343. https://doi.org/10.1002/asi.4630260604

[49] Sergio Segura, Juan C Alonso, Alberto Martin-Lopez, Amador Durán, Javier Troya, and Antonio Ruiz-Cortés. 2022. Automated generation of metamorphic relations for query-based systems. In *Proceedings of the 7th International Workshop on Metamorphic Testing*. 48–55. https://doi.org/10.1145/3524846.3527338

[50] Sergio Segura, Gordon Fraser, Ana B Sanchez, and Antonio Ruiz-Cortés. 2016. A survey on metamorphic testing. *IEEE Transactions on software engineering* 42, 9 (2016), 805–824.

[51] Parikshit Sondhi, Mohit Sharma, Pranam Kolari, and ChengXiang Zhai. 2018. A taxonomy of queries for e-commerce search. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*. 1245–1248. https://doi.org/10.1145/3209978.3210152

[52] Louise T Su. 2003. A comprehensive and systematic model of user evaluation of Web search engines: II. An evaluation by undergraduates. *Journal of the American Society for Information Science and Technology* 54, 13 (2003), 1193–1223. https://doi.org/10.1002/asi.10334

[53] Gareth Terry, Nikki Hayfield, Victoria Clarke, and Virginia Braun. 2017. Thematic analysis. *The SAGE handbook of qualitative research in psychology* 2 (2017), 17–37.

[54] Yuchi Tian, Kexin Pei, Suman Jana, and Baishakhi Ray. 2018. Deeptest: Automated testing of deep-neural-network-driven autonomous cars. In *Proceedings of the 40th international conference on software engineering*. 303–314. https://doi.org/10.1145/3180155.3180220

[55] Christophe Van Gysel. 2017. Remedies against the vocabulary gap in information retrieval. *arXiv preprint arXiv:1711.06004* (2017).

[56] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[57] Liwen Vaughan. 2004. New measurements for search engine evaluation proposed and tested. *Information Processing & Management* 40, 4 (2004), 677–691. https://doi.org/10.1016/s0306-4573(03)00043-8

[58] Wenxuan Wang, Jen-tse Huang, Weibin Wu, Jianping Zhang, Yizhan Huang, Shuqing Li, Pinjia He, and Michael R Lyu. 2023. Mttm: Metamorphic testing for textual content moderation software. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2387–2399. https://doi.org/10.1109/icse48619.2023.00200

[59] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022), 24824–24837.

[60] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE 2023). Association for Computing Machinery*. https://doi.org/10.1109/icse48619.2023.00129

[61] Xiaoyuan Xie, Joshua WK Ho, Christian Murphy, Gail Kaiser, Baowen Xu, and Tsong Yueh Chen. 2011. Testing and validating machine learning classifiers by metamorphic testing. *Journal of Systems and Software* 84, 4 (2011), 544–558. https://doi.org/10.1016/j.jss.2010.11.920

[62] Junjie Ye, Xuanting Chen, Nuo Xu, Can Zu, Zekai Shao, Shichun Liu, Yuhan Cui, Zeyang Zhou, Chao Gong, Yang Shen, et al. 2023. A comprehensive capability analysis of gpt-3 and gpt-3.5 series models. *arXiv preprint arXiv:2303.10420* (2023).

[63] Boxi Yu, Zhiqing Zhong, Xinran Qin, Jiayi Yao, Yuancheng Wang, and Pinjia He. 2022. Automated testing of image captioning systems. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. 467–479. https://doi.org/10.1145/3533767.3534389

[64] Han Zhang, Songlin Wang, Kang Zhang, Zhiling Tang, Yunjiang Jiang, Yun Xiao, Weipeng Yan, and Wen-Yun Yang. 2020. Towards personalized and semantic retrieval: An end-to-end solution for e-commerce search via embedding learning. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2407–2416. https://doi.org/10.1145/3397271.3401446

[65] Mengshi Zhang, Yuqun Zhang, Lingming Zhang, Cong Liu, and Sarfraz Khurshid. 2018. DeepRoad: GAN-based metamorphic testing and input validation framework for autonomous driving systems. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 132–142. https://doi.org/0.1145/3238147.3238187

[66] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. 2022. Automatic chain of thought prompting in large language models. *arXiv preprint arXiv:2210.03493* (2022).

[67] Zhi Quan Zhou, Shaowen Xiang, and Tsong Yueh Chen. 2015. Metamorphic testing for software quality assessment: A study of search engines. *IEEE Transactions on Software Engineering* 42, 3 (2015), 264–284. https://doi.org/10.1109/tse.2015.2478001

Received 2024-02-08; accepted 2024-04-18